# RepoPilot

An AI Software Engineering Agent

*Repository-Aware Code Intelligence | LLM + RAG | AI Agents | Evaluation*

## Project Overview

RepoPilot is a production-style GenAI system that understands, investigates, and helps fix unfamiliar codebases. It combines LLM + RAG, AI Agents, Evaluation, and Backend Engineering into a comprehensive AI software engineering tool.

Given a GitHub repository with hundreds of files, RepoPilot can answer questions like:

- How does authentication work in this project?
- Why does this API return a 500 error?
- Which files would I need to change to add a new feature?
- Find the tests related to this function.
- Suggest a fix for this bug and verify it by running the relevant tests.

RepoPilot inspects the actual repository, retrieves relevant code, calls tools to investigate problems, and produces answers backed by file paths and line numbers. For bugs, it can propose patches and run approved tests in an isolated environment.

## 1. Why This Is a Strong Choice for AI Engineer Placements

Current job requirements from Google, OpenAI, and Anthropic align with the skills this project demonstrates:

| Skill | What the Implementation Demonstrates |
|---|---|
| LLM Development | Prompting, structured output, model integration |
| RAG | Finding relevant code and documentation |
| Agentic Workflows | Selecting and calling tools to investigate bugs |
| Information Retrieval | Hybrid search, reranking, retrieval evaluation |
| AI Reliability | Citations, abstention, hallucination tests |
| AI Evaluation | Benchmarks, regression testing, comparisons |
| Software Engineering | API design, security, deployment, automated tests |

# 2. Core Modules

## Module A - Repository Intelligence

The user submits a public GitHub repository. The system indexes its source code, README files, and documentation. It answers questions with links to the relevant files and line ranges.

Core concepts: AST-based code chunking, embeddings, hybrid search, citations.

## Module B - Bug Investigation Agent

Given a bug report, it searches for relevant functions, traces dependencies, locates tests, and produces a proposed fix. The initial implementation suggests patches rather than autonomously rewriting the repository.

Core concepts: Tool calling, workflow orchestration, state management, controlled execution.

## Module C - AI Quality Dashboard

Shows which questions the system answers correctly, whether citations are valid, response latency, and query cost. Compares different retrieval configurations and model choices.

Core concepts: Recall@k, MRR, groundedness, latency, cost, and automated regression tests.

# 3. Technical Architecture

The system is organized into five layers:

### Layer 1: React Frontend

- Repository input and management
- Chat interface with code citations
- Evaluation dashboard with metrics visualization

### Layer 2: Python + FastAPI Backend

- Authentication and request routing
- API and workflow orchestration

### Layer 3: Retrieval Pipeline

- AST parsing with Tree-sitter
- BM25 lexical search
- Vector search with embeddings
- Optional reranking

### Layer 4: Agent Tools

- Symbol search and file reading
- Test discovery and sandboxed test runs

### Layer 5: Storage and Model Layer

- Qdrant / PostgreSQL with pgvector for vectors
- LLM API with provider-independent interface
- Structured logging and evaluation storage

### Technology Stack

| Component | Technology |
|---|---|
| Frontend | React + Vite + CSS |
| Backend | Python + FastAPI |
| Code Parsing | Tree-sitter |
| Search | BM25 + vector retrieval + optional reranker |
| Vector Storage | Qdrant or PostgreSQL with pgvector |
| LLM | Hosted model, provider-independent interface |
| Agent Workflow | Python functions; LangGraph if needed |
| Testing | Pytest + GitHub Actions |
| Deployment | Docker + hosted application service |

## 4. Research Component

Research Question: "Does syntax-aware chunking combined with hybrid retrieval improve code-question answering compared with conventional vector-only RAG?"

### Experimental Baselines

| Experiment | Configuration |
|---|---|
| Baseline A | Fixed-size chunks + vector search |
| Baseline B | Function/class-aware chunks + vector search |
| Proposed System | Function/class-aware chunks + BM25 + vector search + reranking |

### Evaluation Metrics

- Retrieval: Recall@5 and Mean Reciprocal Rank (MRR)
- Answer quality: Correctness, relevance, evidence support
- Citation quality: Whether cited files/lines support the claim
- Reliability: Correctly declining unanswerable questions
- Efficiency: Median latency, token usage, cost per query

# 5. Implementation Phases

## Phase 1 - Working RAG System

Ingest one public repository, parse files into meaningful chunks, index them, and answer questions with file-and-line citations. Implement the React UI and FastAPI service.

## Phase 2 - Improve Retrieval

Add lexical search, hybrid retrieval, and optionally reranking. Run initial benchmark and compare each configuration.

## Phase 3 - Add Debugging Workflow

Give the agent a small set of controlled tools. It should inspect the repository, find relevant tests, propose a patch, and run only explicitly approved commands in an isolated environment.

## Phase 4 - Make It Portfolio-Ready

Add automated evaluation and regression tests, structured logs, latency/cost tracking, Docker deployment, a clean README, an architecture diagram, and a short recorded demo.

# 6. Resume Entry Template

**RepoPilot - Repository-Aware AI Software Engineering Agent**

*Python, FastAPI, React, RAG, Vector Search, LLMs*

- Built an LLM-powered codebase assistant using AST-aware chunking and hybrid retrieval to answer repository questions with source-level citations.
- Developed a tool-using debugging workflow to identify relevant code, locate tests, and generate candidate patches with controlled test execution.
- Designed an evaluation benchmark of [N] questions and compared retrieval pipelines, achieving [measured Recall@5 / MRR] while tracking citation validity, latency, and inference cost.
- Implemented automated regression tests and containerized deployment to improve reproducibility and reliability.

# 7. What to Avoid

- Generic PDF chatbots
- Simple LLM API wrappers
- Multi-agent systems that only pass text between agents
- Training your own LLM (focus on retrieval quality, tool design, evaluation, security, and deployment)

*This project complements traditional ML/CV projects by demonstrating modern LLM systems engineering.*